

Sistemi di Calcolo (A.A. 2024-2025)

Corso di Laurea in Ingegneria Informatica e Automatica
Sapienza Università di Roma



Compito (23/09/2025) – Durata 1h 30'

Inserire nome, cognome e matricola nel file `studente.txt`.

ISTRUZIONI PER STUDENTI DSA: svolgere a scelta due parti su tre.

Parte 1 (programmazione IA32)

La funzione `lower` converte tutte le lettere contenute in una stringa in caratteri minuscoli. Questa versione incompleta della funzione non riconosce però numeri, punteggiatura o qualsiasi altro carattere che non sia una lettera o uno spazio. Nel caso trovi un carattere sconosciuto la funzione smette di elaborare la stringa e ritorna l'indice del carattere sconosciuto, incrementato di una unità; se invece riesce ad elaborare l'intera stringa, ritorna 0. Nella directory E1, si traduca in assembly IA32 la seguente funzione C scrivendo un modulo `e1A.s`:

```
unsigned int lower(char *data, unsigned int len)
{
    for (unsigned int i = 0; i < len; i++){
        if (!is_known(data[i]))
            return i+1;
        if (data[i]>=97 && data[i]<123) //lower letter
            continue;
        if (data[i]==32) //space
            continue;
        data[i] += 32; // lower to upper
    }
    return 0;
}
```

L'unico criterio di valutazione è la correttezza. Generare un file eseguibile `e1A` con `gcc -m32 -g` (per evitare possibili warning è possibile aggiungere lo switch `-no-pie`). Per i test, compilare il programma insieme al programma di prova `e1A_main.c`.

Non modificare in alcun modo `e1A_main.c`. Prima di tradurre il programma in IA32 si suggerisce di scrivere nel file `e1A_eq.c` una versione C equivalente più vicina all'assembly.

Parte 2 (programmazione di sistema POSIX)

Si scriva una funzione che conti le occorrenze di un carattere all'interno di `n` sottostringhe consecutive generate da una stringa in input. Nello specifico, si scriva nel file `E2/e2A.c` una funzione con il seguente prototipo (definito in `E2/e2A.h`):

```
int countOccurrencesParallel(const char* s, char c, int n);
```

che, data una stringa `s`, un carattere `c` e intero `n`, (1) divida la stringa in `n` sottostringhe consecutive di lunghezza uguale, (2) conti il numero di occorrenze del carattere `c` in ogni sottostringa e (3) ritorni l'indice della sottostringa che possiede il numero di occorrenze massimo. Il conteggio delle occorrenze deve essere svolto in parallelo sfruttando i **thread**, e

quindi assegnando le n sottostringhe consecutive a n thread distinti. Se la lunghezza della stringa non è divisibile per n , i caratteri finali della stringa, in numero pari al resto della divisione per n , vengono tutti assegnati all'ultimo thread.

Ad esempio, supponiamo di avere una stringa s da 39 caratteri:

```
s = "Shall I compare thee to a summer's day?"
```

Una ipotetica invocazione potrebbe essere:

```
countOccurrencesParallel(s, "e", 4)
```

In questo caso, la funzione dividerebbe la stringa in 4 sottostringhe consecutive, tuttavia, si noti che la lunghezza della stringa non è divisibile per 4. Data la divisione $39/4=9.75$, possiamo creare sottostringhe di lunghezza 9 e l'ultima avrà lunghezza $9 + 3$ (il resto della divisione $39/4$). Le stringhe che avremmo sarebbero:

1. "Shall I c" (9 caratteri) che verrà processata dal thread #0 con risultato 0 dato che non contiene "e";
2. "ompare th" (9 caratteri) che verrà processata dal thread #1 con risultato 1 dato che contiene una "e";
3. "ee to a s" (9 caratteri) che verrà processata dal thread #2 con risultato 2 dato che contiene 2 "e";
4. "ummer's day?" (11 caratteri) che verrà processata dal thread #3 con risultato 1 dato che contiene una "e";

La funzione `countOccurrencesParallel` ritornerà quindi 2, ovvero l'indice della sottostringa che ha restituito il numero di occorrenze maggiore.

Nota: nel caso in cui il massimo numero di occorrenze sia uguale per più sottostringhe, ritornare l'indice più basso. Nei casi limite in cui la stringa sia vuota o n sia uguale a 0 la funzione deve ritornare -1.

Nota: Si ricorda di utilizzare il flag **-lpthread** per la compilazione quando si utilizzano i thread POSIX.

Suggerimento: per passare a un thread più di un argomento si può definire e utilizzare una struttura dati e poi passare al thread un puntatore alla stessa.

Per i test, compilare il programma insieme al programma di prova `e2A_main.c` fornito, che **non** deve essere modificato.

Parte 3 (quiz)

Si risponda ai seguenti quiz, inserendo le risposte (A, B, C, D o E per ogni domanda) nel file `e3A.txt`. Una sola risposta è quella giusta. Rispondere E equivale a non rispondere (0 punti).

Domanda 1 (ottimizzazioni)

Cosa fa il *constant folding*?

A	Sostituisce espressioni costanti con il risultato	B	Sposta variabili in registri
C	Elimina cicli	D	Riduce la dimensione degli array

Motivare la risposta nel file `M1.txt`. **Risposte non motivate saranno considerate nulle.**

Domanda 2 (thread)

Si consideri il seguente codice

```
int global_int = 0;

void *job(void * arg){
    int local_int;
    local_int = global_int;
    printf("global is %d", local_int);
    global_int = local_int + 1;
    return NULL;
}

int main(){
    pthread_t threads[4];
    for(int thread_i = 0; thread_i < 4; thread_i++){
        if (pthread_create(&threads[i], NULL, job, NULL) != 0)
            exit(EXIT_FAILURE);
    }
    printf("Ok");
    return 0;
}
```

Quale dei seguenti output non potrebbe essere generato dal programma sopra elencato?

A	0 1 2 3 Ok	B	0 1 Ok
C	0 1 3 OK	D	0 1 2 1 Ok

Motivare la risposta nel file M2.txt. **Risposte non motivate saranno considerate nulle.**

Domanda 3 (debugging)

Dato il seguente listato C:

```
1: void foo() {
2:     char *p = malloc(100);
3:     strcpy(p , "YYYYYYYYYYYYYYYYYYYYYYYY");
4: }
5: int main() {
6:     foo();
7:     return 0;
8: }
```

Compilando il programma per architettura x86 in un eseguibile chiamato main, un'analisi dell'esecuzione sotto Valgrind riporta:

```
==10255== Memcheck, a memory error detector
==10255== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==10255== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright
info
==10255== Command: ./foo
```

```

==10255==
==10255==
==10255== HEAP SUMMARY:
==10255==      in use at exit: 100 bytes in 1 blocks
==10255==    total heap usage: 1 allocs, 0 frees, 100 bytes allocated
==10255==
==10255== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==10255==    at 0x4840664: malloc (in
/usr/libexec/valgrind/vgpreload_memcheck-x86-linux.so)
==10255==    by 0x1091A9: foo (foo.c:2)
==10255==    by 0x1091F6: main (foo.c:6)
==10255==
==10255== LEAK SUMMARY:
==10255==    definitely lost: 100 bytes in 1 blocks
==10255==    indirectly lost: 0 bytes in 0 blocks
==10255==    possibly lost: 0 bytes in 0 blocks
==10255==    still reachable: 0 bytes in 0 blocks
==10255==    suppressed: 0 bytes in 0 blocks
==10255==
==10255== For lists of detected and suppressed errors, rerun with: -s
==10255== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Di cosa ci informa valgrind?

A	heap overrun	B	use-after-free
C	memory leak	D	non riporta nessun errore

Motivare la risposta nel file M3.txt. **Risposte non motivate saranno considerate nulle.**

Domanda 4 (cache)

Si consideri una cache completamente associativa con 4 linee da 64 byte ciascuna e politica di rimpiazzo LRU, inizialmente vuota. Potendo scegliere fra più linee vuote, si usa la linea con indice più basso. Si ha inoltre un processo che accede in sequenza ai seguenti indirizzi di memoria (senza interruzioni): 6200, 413, 62, 42, 916, 400, 520.

Alla fine della sequenza di accessi, quali sono gli indici dei blocchi contenuti nelle 4 linee di cache? Il trattino indica che la linea di cache rimane vuota.

A	8, 6, 0, 14	B	8, 6, 0, -
C	96, 6, 0, 14	D	6, 96, -, -

Motivare la risposta nel file M4.txt. **Risposte non motivate saranno considerate nulle.**